VHDL 실습 과제 2번 보고서

래치와 플립플롭 5종 - Logisim 게이트 회로, VHDL 코드, Questa 시뮬레이션

1. 5개 회로도 전체 요약

이번 실습에서는 1비트를 기억하는 기본 소자 5가지를 Logisim에서 게이트 수준으로 직접 그리고, 같은 소자를 VHDL로 작성하여 두 구현의 출력이 같은지 확인하였다. 각 스크린샷의 위쪽은 게이트로 직접 그린 회로이고, 아래쪽은 VHDL 엔티티를 Logisim 블록으로 불러온 것이다. 이어서 같은 VHDL 코드를 Questa Altera Starter FPGA Edition 2025.2 에서 vcom 으로 컴파일하고 vsim 으로 시뮬레이션하여, 게이트 회로·Logisim VHDL 블록·Questa 파형 세 가지 결과가 모두 같은지 비교하였다.

번호	회로	Logisim 회로 구성	VHDL 파일	핵심 동작
1	RS 래치	NOR 게이트 2개의 출력을 서로 상대 입력으로 교차 연결	rs_latch.vhd	S=1 Set, R=1 Reset, 둘 다 0이면 유지, 둘 다 1은 금지
2	D 래치	NAND 게이트 4개로 D와 en을 받는 래치	d_latch.vhd	en=1이면 Q가 D를 따라가고, en=0이면 이전 값 유지
3	D 플립플롭	D 래치 2개 + NOT 게이트로 Master-Slave 구성	d_ff.vhd	clk가 0→1로 바뀌는 순간에만 D를 저장
4	JK 플립플롭	NAND 게이트로 d_next 논리를 만들고 출력을 입력으로 피드백	jk_ff.vhd	상승 에지에서 00 유지, 01 Reset, 10 Set, 11 반전
5	비동기 Set/Clear D 플립플롭	D 플립플롭에 클록과 무관한 set, clr 경로 추가	d_ff_async.vhd	clr=1이면 즉시 0, set=1이면 즉시 1, 둘 다 1이면 Clear 우선

- 1번, 2번(래치)은 enable 레벨 동안 입력이 그대로 출력에 반영되는 레벨 트리거 소자이다.
- 3번, 4번, 5번(플립플롭)은 클록 상승 에지 순간의 입력만 저장하는 에지 트리거 소자이다.
- 5번은 여기에 클록을 무시하는 비동기 set/clr 입력을 더한 것으로, VHDL에서는 감지 목록과 if 검사 순서로 우선순위를 정한다.
- Questa 에서는 vlib work → vcom 파일.vhd → vsim work.엔티티 순서로 로드한 뒤, add wave 로 신호를 등록하고 force/run 명령을 10 ns 단위로 주어 파형을 얻었다. 플립플롭은 입력을 바꾼 뒤 clk 를 0→1 로 올리는 순서로 상승 에지를 만들었다.

2. RS 래치 (rs_latch)

NOR 게이트 2개를 교차 결합하여 S, R 입력으로 1비트를 Set/Reset하고 기억하는 가장 기본적인 래치이다.

■ VHDL 코드

[rs_latch.vhd]

```
library ieee;
use ieee.std_logic_1164.all;

entity rs_latch is
    port (
        s, r : in std_logic;
        q, q_bar : out std_logic
    );
end rs_latch;

architecture gate_level of rs_latch is
    signal qi, qbi : std_logic;
begin
    qi  <= not (r or qbi);
    qbi <= not (s or qi);
    q <= qi;
    q_bar <= qbi;
end gate_level;
```

■ 코드 설명

- process 없이 동시 신호 할당문 두 줄로 NOR 게이트 2개를 그대로 기술하였다.
- qi와 qbi가 서로의 입력으로 들어가는 피드백 구조가 기억 기능을 만든다.
- q_bar는 두 번째 NOR의 출력이므로 S=R=1(금지 입력)이면 q와 q_bar가 모두 0이 된다.

■ 회로도와 실행 결과

회로도 위쪽은 r을 받는 NOR와 s를 받는 NOR의 출력이 서로 상대 게이트의 입력으로 되돌아가는 구조이고, 아래쪽은 같은 입력을 받는 rs_latch VHDL 블록이다. Set → 유지 → Reset → 유지 → 금지 입력 순서로 입력을 바꾸며 두 구현의 q, q_bar를 비교하였다.

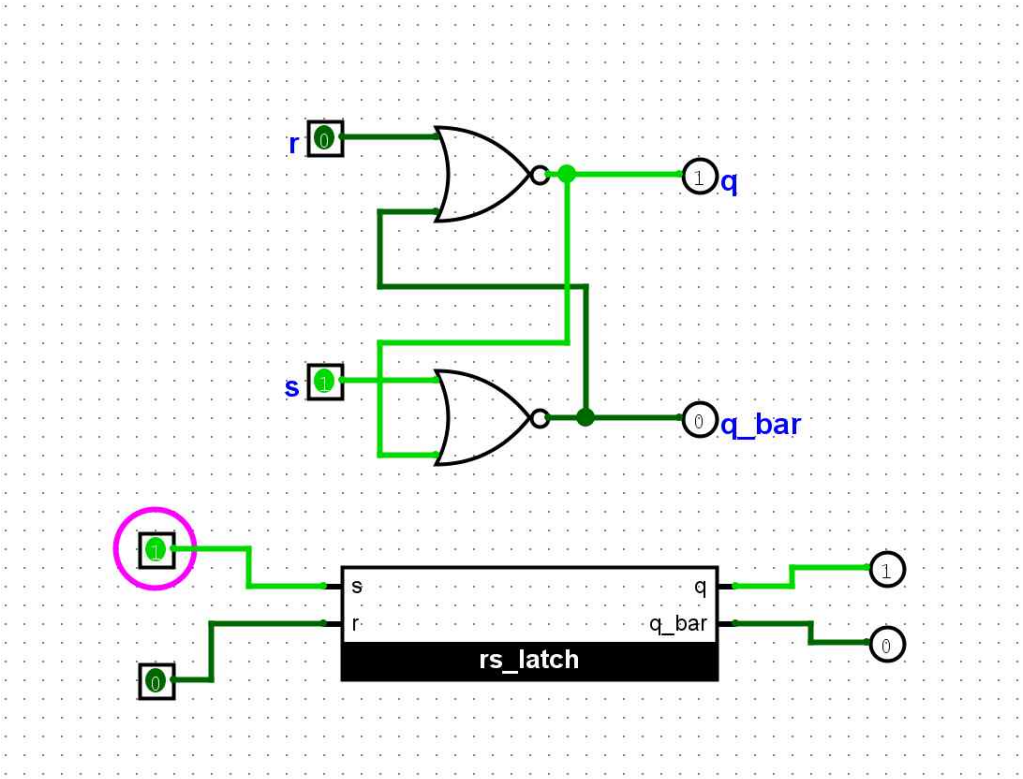


그림 1-1. S=1, R=0 → q=1, q_bar=0 (Set). 게이트 회로와 rs_latch 블록 모두 q=1

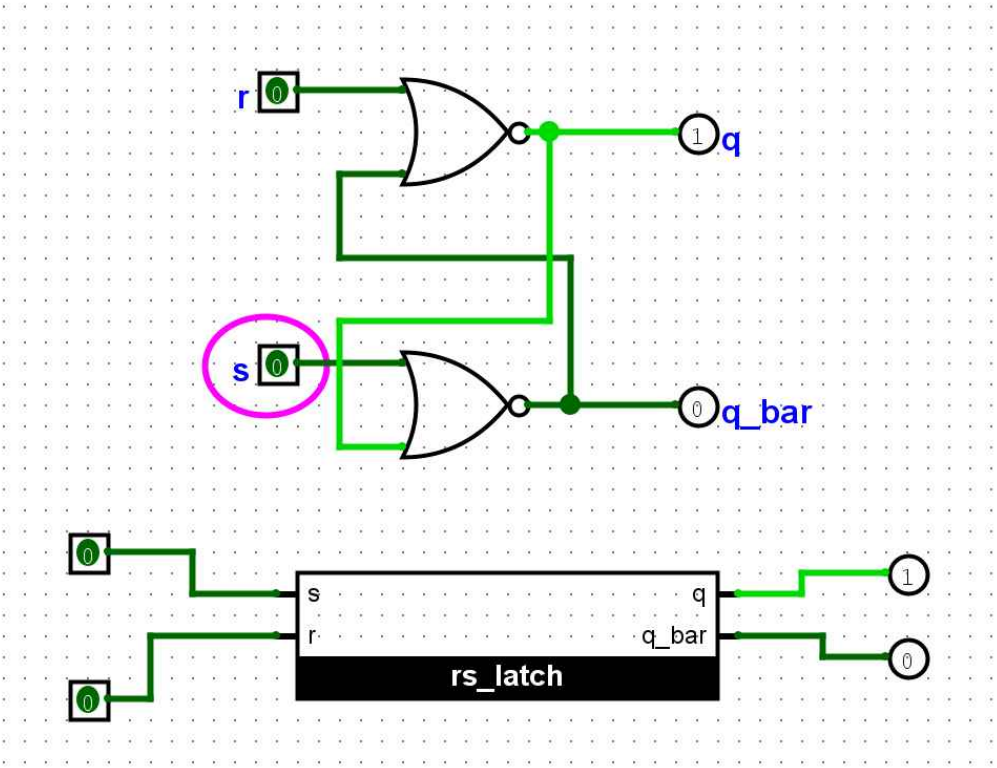


그림 1-2. S=0, R=0 → q=1 유지. 입력을 모두 내려도 이전 값 1을 기억

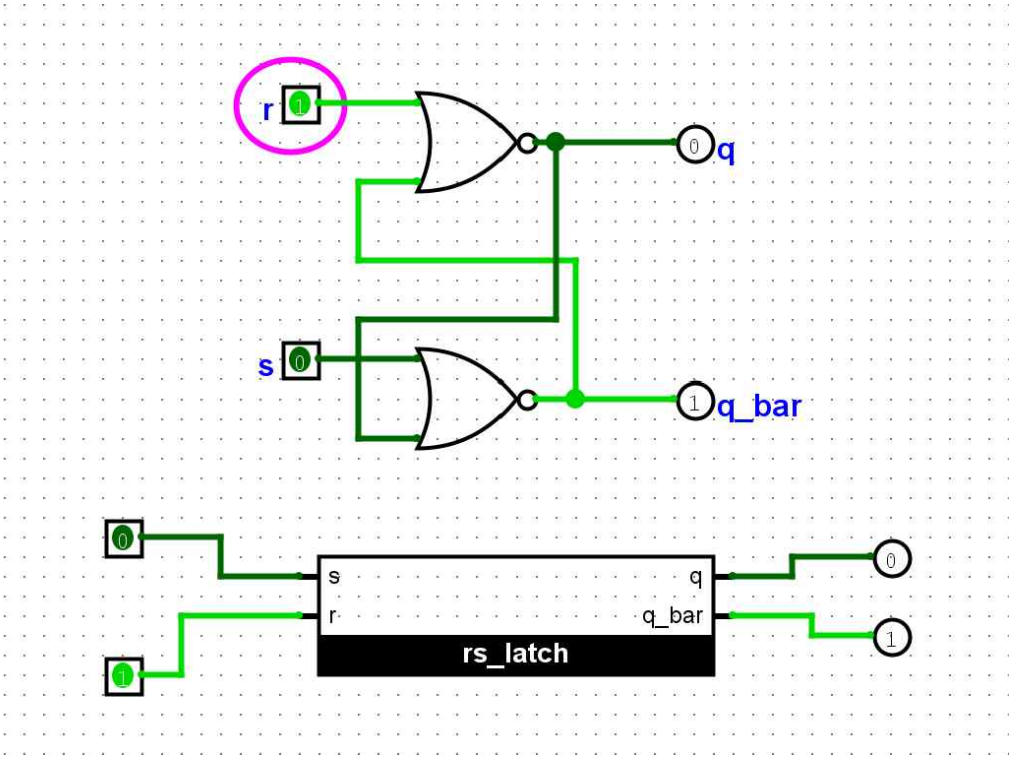


그림 1-3. S=0, R=1 → q=0, q_bar=1 (Reset)

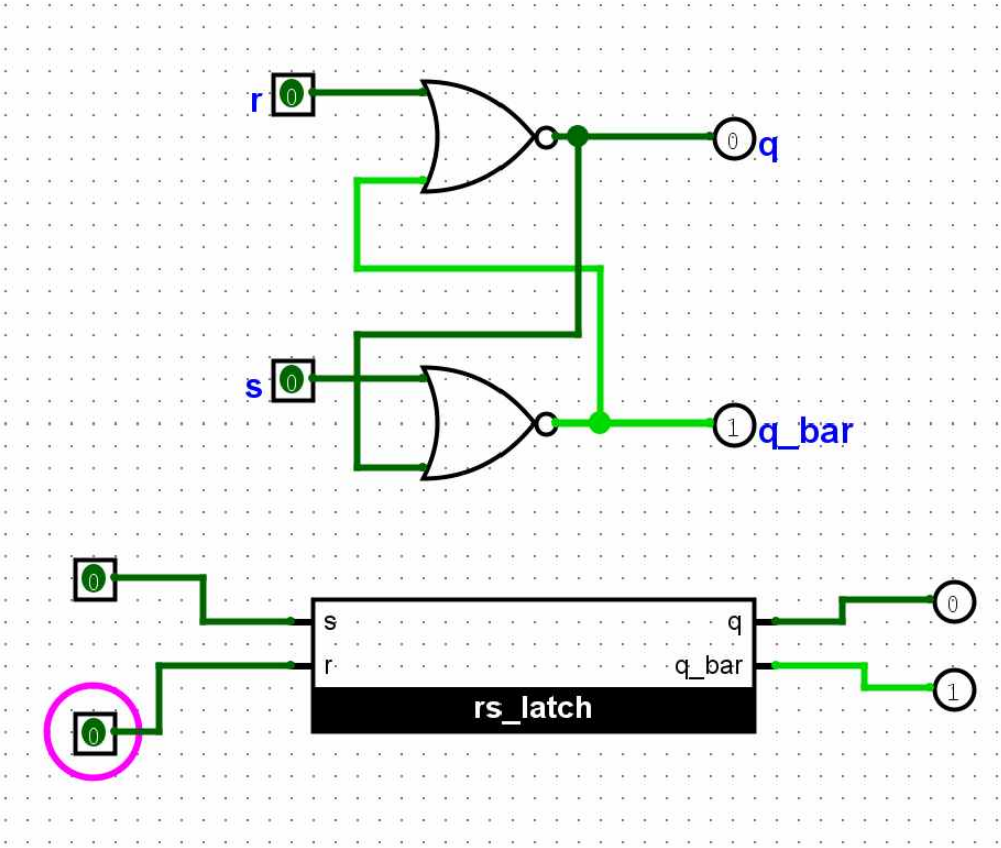


그림 1-4. S=0, R=0 → q=0 유지. Reset 이후의 0을 그대로 기억

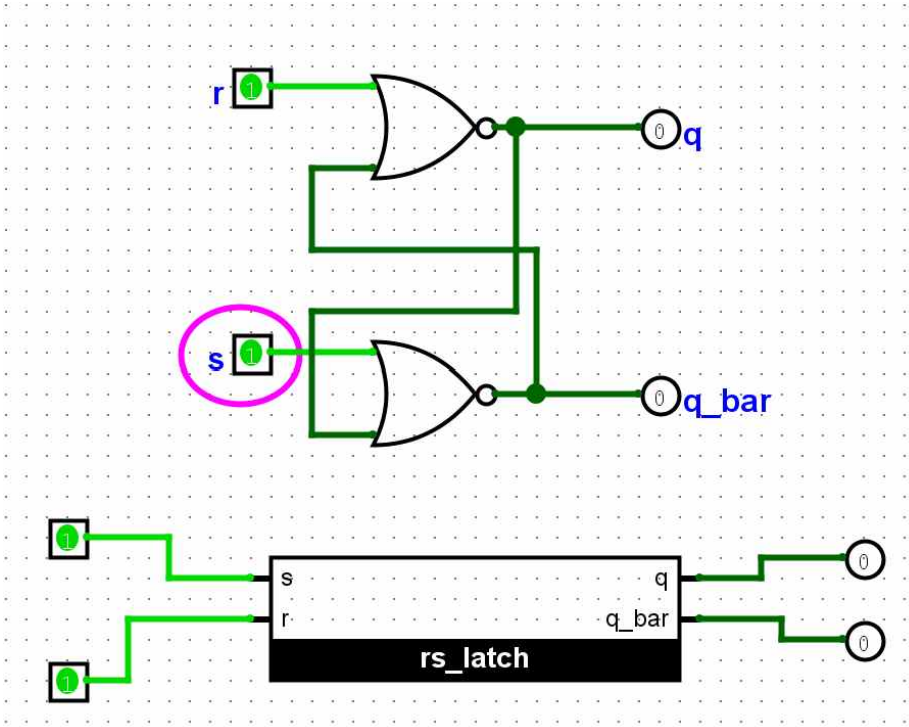


그림 1-5. S=1, R=1 → q=0, q_bar=0 (금지 입력). 두 출력이 상보 관계를 잃음

다섯 단계 모두 게이트 회로와 VHDL 블록의 출력이 같았고, S=R=0에서 직전 값이 유지되는 기억 동작과 S=R=1의 금지 상태를 확인하였다.

■ Questa 컴파일과 시뮬레이션

같은 rs_latch.vhd 를 Questa Altera Starter FPGA Edition 2025.2 에서 vcom 으로 컴파일한 뒤, vsim 에서 force 명령으로 Logisim 실험과 같은 순서의 입력을 주고 파형을 확인하였다. 아래는 Transcript 에 입력한 시뮬레이션 스크립트이다.

[rs_latch_sim.do]

```
add wave sim:/rs_latch/s sim:/rs_latch/r sim:/rs_latch/q sim:/rs_latch/q_bar
force s 1
force r 0
run 10 ns
force s 0
run 10 ns
force r 1
run 10 ns
force r 0
run 10 ns
force s 1
force r 1
run 10 ns
wave zoom full
```

```
VSIM 18> vcom {C:\VHDL_Assignment\2번실습\rs_latch.vhd}
# Questa Altera Starter FPGA Edition-64 vcom 2025.2 Compiler 2025.05 May 31 2025
# Start time: 09:29:19 on Nov 11,2026
# vcom -reportprogress 300 C:\VHDL_Assignment\2¹ø½Ç½À\rs_latch.vhd
# ** Note: (vcom-220) 'C:/Users/gahyu/AppData/Local/quartus/Q1_rs_latch.mpf' is used as the ini file.
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity rs_latch
# -- Compiling architecture gate_level of rs_latch
# End time: 09:29:19 on Nov 11,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

그림 1-6a. rs_latch.vhd 컴파일 결과. Errors: 0, Warnings: 0 으로 오류 없이 컴파일됨

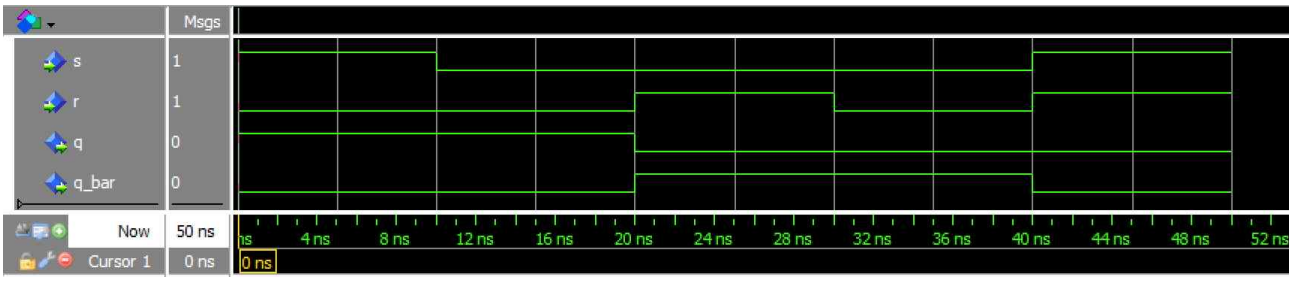


그림 1-6b. Questa 파형. 위에서부터 s, r, q, q_bar. 각 구간은 10 ns

구간	입력	q	확인한 동작
0~10 ns	s=1, r=0	1	Set. q=1, q_bar=0 (그림 1-1과 동일)
10~20 ns	s=0, r=0	1	유지. 입력을 내려도 1을 기억 (그림 1-2)
20~30 ns	s=0, r=1	0	Reset. q=0, q_bar=1 (그림 1-3)
30~40 ns	s=0, r=0	0	유지. Reset 이후의 0을 기억 (그림 1-4)
40~50 ns	s=1, r=1	0	금지 입력. q와 q_bar가 모두 0 (그림 1-5)

다섯 구간의 q, q_bar 값이 Logisim 게이트 회로 및 VHDL 블록의 결과(그림 1-1 ~ 1-5)와 모두 같았다. 세 구현이 동일한 RS 래치 기능을 나타낸다.

3. D 래치 (d_latch)

NAND 게이트 4개로 만든 래치로, en=1인 동안 출력 Q가 입력 D를 그대로 따라가고 en=0이면 마지막 값을 유지한다.

■ VHDL 코드

[d_latch.vhd]

```
library ieee;
use ieee.std_logic_1164.all;

entity d_latch is
    port (
        d, en : in std_logic;
        q, q_bar : out std_logic
    );
end d_latch;

architecture rtl of d_latch is
    signal qi : std_logic;
begin
    process(d, en)
    begin
        if en = '1' then
            qi <= d;
        end if;
    end process;

    q <= qi;
    q_bar <= not qi;
end rtl;
```

■ 코드 설명

- process(d, en)에 클록이 없고 if en = '1' then qi <= d; 만 있으며 else 분기가 없다.
- else가 없으므로 en=0일 때 qi는 이전 값을 유지하고, 이것이 레벨 감지 래치가 된다.
- q_bar는 qi의 반전으로 만들어 항상 q와 상보 관계를 유지한다.

■ 회로도와 실행 결과

회로도 위쪽은 D와 en을 NAND한 S_bar, D_bar와 en을 NAND한 R_bar가 뒤쪽 NAND 래치(Q, Q_bar)로 들어가는 구조이고, 아래쪽은 d_latch VHDL 블록이다. en을 열고 닫으면서 D를 바꾸어 출력이 언제 따라가고 언제 유지되는지 확인하였다.

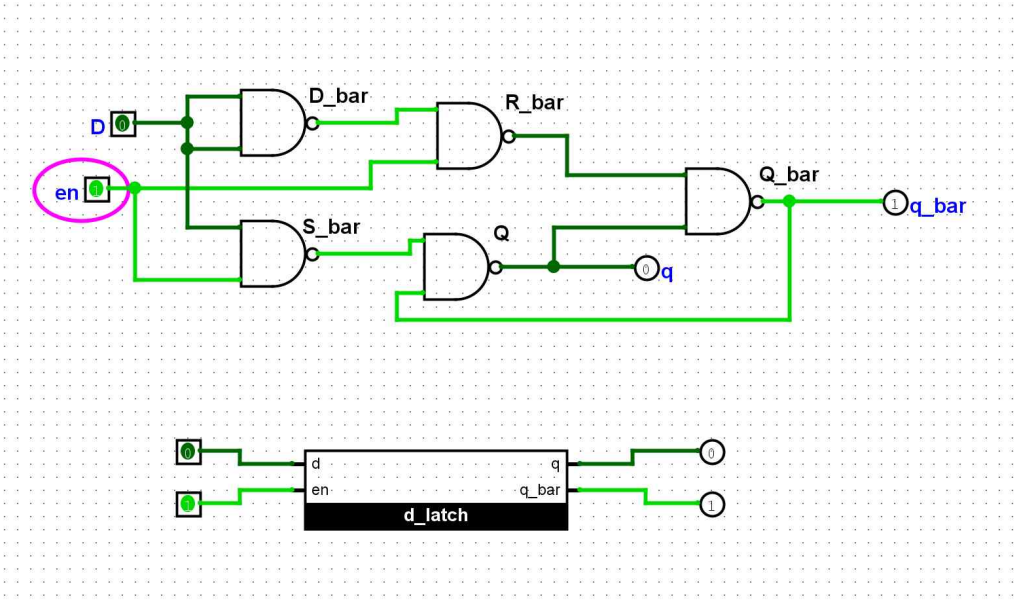


그림 2-1. en=1, D=0 → q=0. en이 열린 상태라 D=0이 바로 출력에 반영

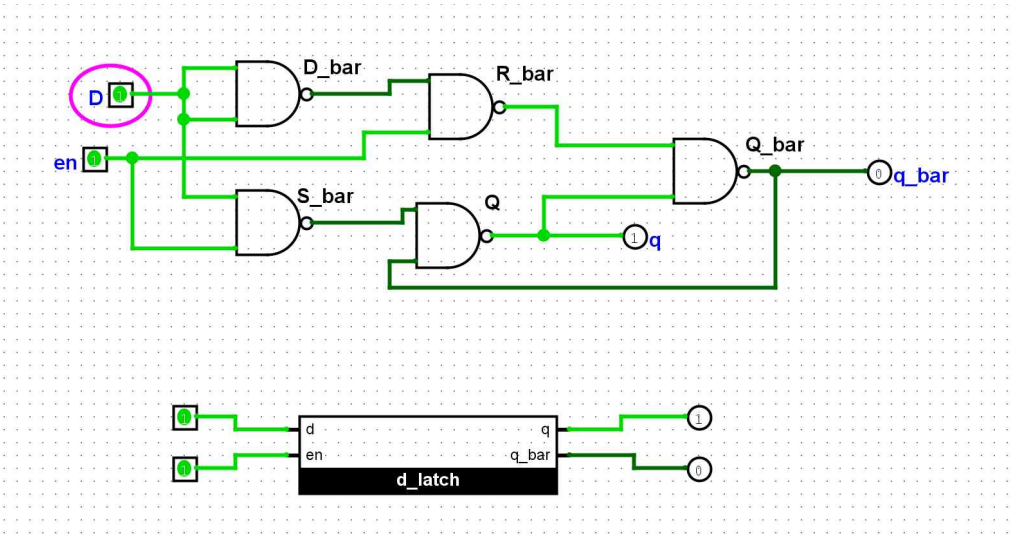


그림 2-2. en=1, D=1 → q=1. 래치가 열려 있어 Q가 D를 따라감

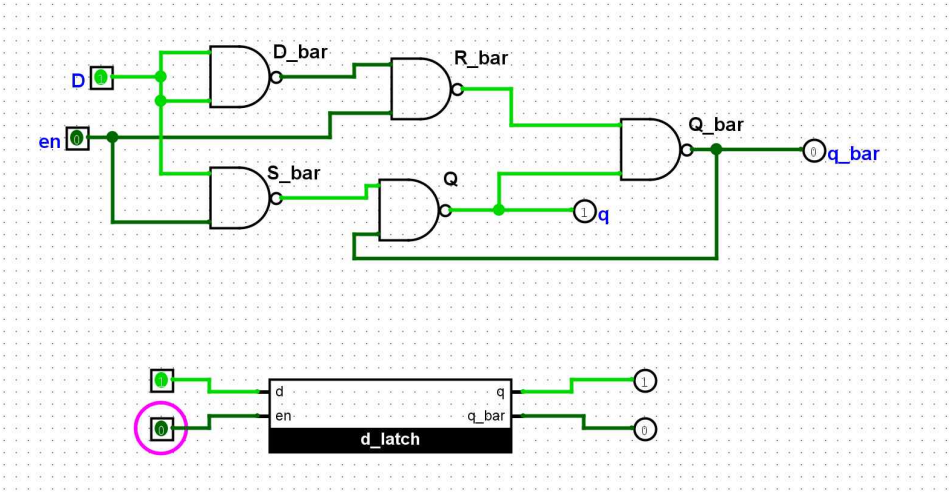


그림 2-3. en=0, D=1 → q=1 유지. en을 내리는 순간의 D 값이 저장됨

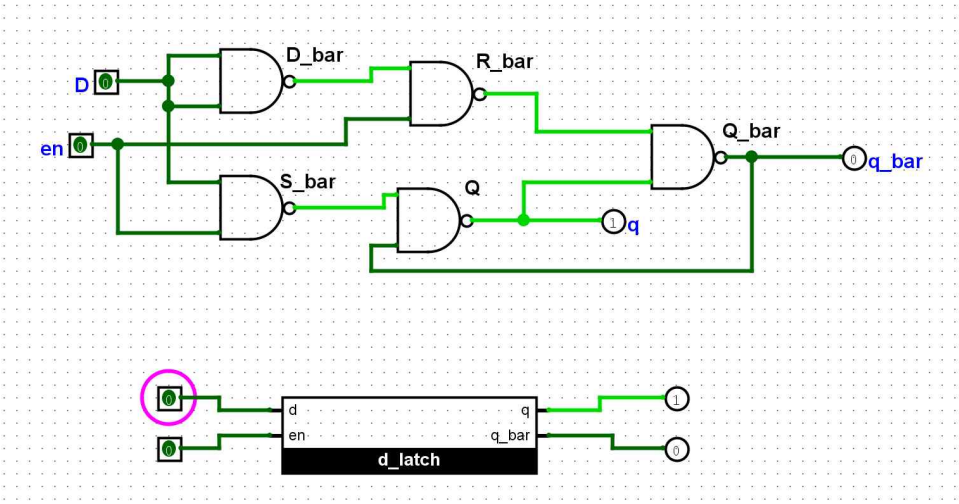


그림 2-4. en=0, D=0 → q=1 유지. 닫힌 상태에서는 D가 바뀌어도 출력 불변

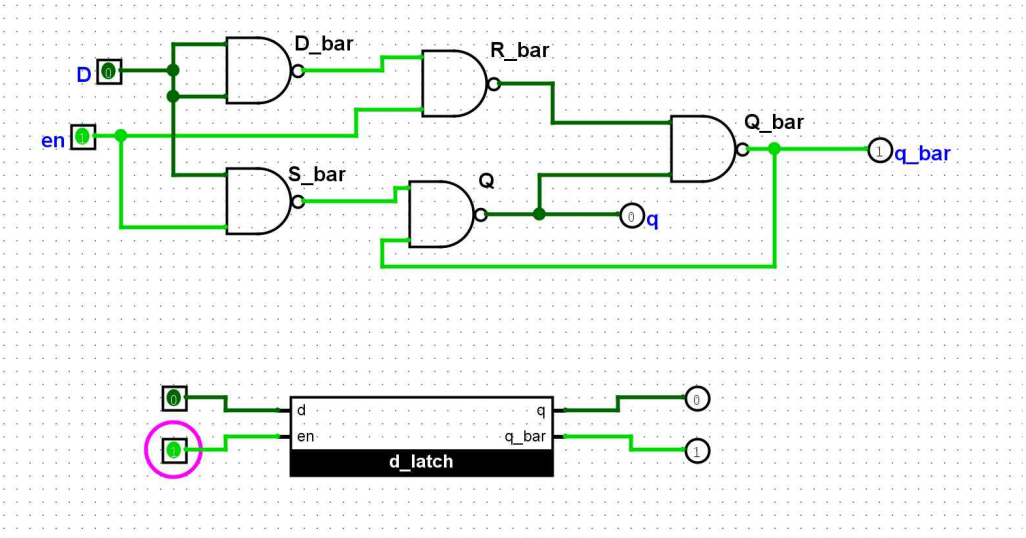


그림 2-5. en=1, D=0 → q=0. 다시 열리자 현재 D=0이 반영됨
en=1 구간에서는 Q가 D를 즉시 따라가고 en=0 구간에서는 D를 바꾸어도 Q가 변하지 않았으며, 게이트 회로와 VHDL 블록의 출력이 모두 일치하였다.

■ Questa 컴파일과 시뮬레이션

같은 d_latch.vhd 를 Questa Altera Starter FPGA Edition 2025.2 에서 vcom 으로 컴파일한 뒤, vsim 에서 force 명령으로 Logisim 실험과 같은 순서의 입력을 주고 파형을 확인하였다. 아래는 Transcript 에 입력한 시뮬레이션 스크립트이다.

[d_latch_sim.do]

```
add wave sim:/d_latch/d sim:/d_latch/en sim:/d_latch/q sim:/d_latch/q_bar
force en 1
force d 0
run 10 ns
force d 1
run 10 ns
force en 0
run 10 ns
force d 0
run 10 ns
force en 1
run 10 ns
wave zoom full
```

```
VSIM 63> vcom {C:\VHDL_Assignment\test2\d_latch.vhd}
# Questa Altera Starter FPGA Edition-64 vcom 2025.2 Compiler 2025.05 May 31 2025
# Start time: 09:41:31 on Nov 11,2026
# vcom -reportprogress 300 C:\VHDL_Assignment\test2\d_latch.vhd
# ** Note: (vcom-220) 'C:/Users/gahyu/AppData/Local/quartus/Q1_rs_latch.mpf' is used as the ini file.
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity d_latch
# -- Compiling architecture rtl of d_latch
# End time: 09:41:31 on Nov 11,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

그림 2-6a. d_latch.vhd 컴파일 결과. Errors: 0, Warnings: 0 으로 오류 없이 컴파일됨

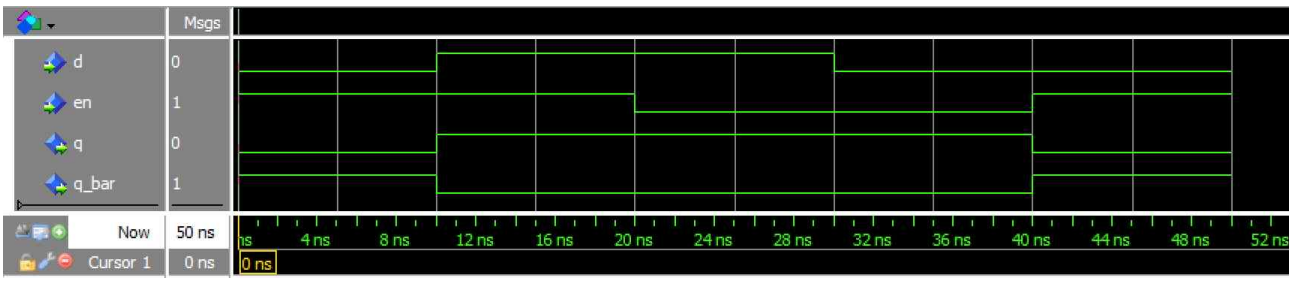


그림 2-6b. Questa 파형. 위에서부터 d, en, q, q_bar. 각 구간은 10 ns

구간	입력	q	확인한 동작
0~10 ns	en=1, d=0	0	열린 상태. d=0이 바로 반영 (그림 2-1)
10~20 ns	en=1, d=1	1	열린 상태. q가 d를 따라감 (그림 2-2)
20~30 ns	en=0, d=1	1	닫힘. en을 내리는 순간의 d=1 저장 (그림 2-3)
30~40 ns	en=0, d=0	1	닫힘. d를 바꿔도 q 불변 (그림 2-4)
40~50 ns	en=1, d=0	0	다시 열림. 현재 d=0 반영 (그림 2-5)

en=1 구간에서만 q가 d를 따라가고 en=0 구간에서는 d 변화가 무시되었다. 다섯 구간 모두 그림 2-1 ~ 2-5의 Logisim 결과와 일치한다.

4. D 플립플롭 (d_ff)

2번 D 래치 2개를 NOT 게이트로 반전한 클록과 함께 Master-Slave로 연결하여, 클록 상승 에지에서만 D를 저장하는 플립플롭이다.

■ VHDL 코드

[d_ff.vhd]

```
library ieee;
use ieee.std_logic_1164.all;

entity d_ff is
    port (
        d, clk : in std_logic;
        q, q_bar : out std_logic
    );
end d_ff;

architecture rtl of d_ff is
    signal qi : std_logic;
begin
    process(clk)
    begin
        if rising_edge(clk) then
            qi <= d;
        end if;
    end process;

    q <= qi;
    q_bar <= not qi;
end rtl;
```

■ 코드 설명

- if rising_edge(clk) then qi <= d; 로 클록 상승 에지에서만 d를 저장한다.
- 게이트 회로에서 래치 2개와 NOT 게이트로 만든 에지 동작을 VHDL에서는 rising_edge() 한 줄로 표현한다.
- d는 에지 순간에만 샘플링되므로 감지 목록에 넣지 않는다.

■ 회로도와 실행 결과

회로도 위쪽은 master 래치(en = NOT clk)와 slave 래치(en = clk)가 직렬로 연결된 구조이고, 아래쪽은 d_ff VHDL 블록이다. clk=0일 때 master가 D를 받아들이고, clk가 0→1로 바뀌는 순간 slave가 열리면서 출력 q가 바뀐다.

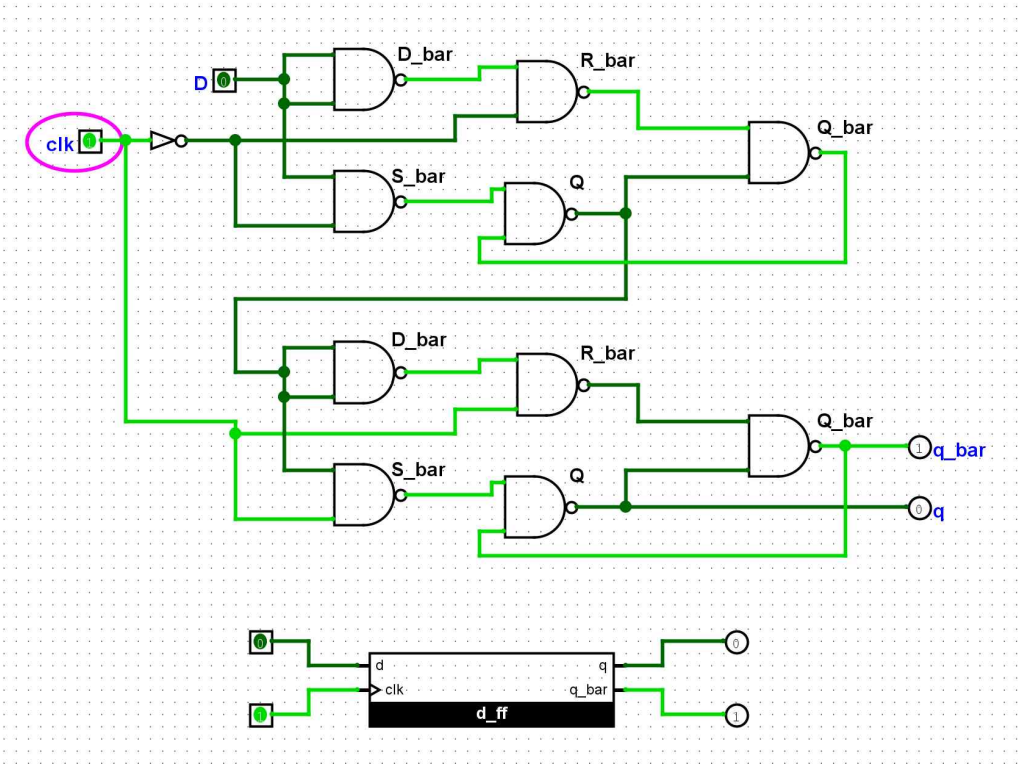


그림 3-1. D=0 상태에서 clk 0→1 → q=0. 상승 에지에서 D=0이 저장됨

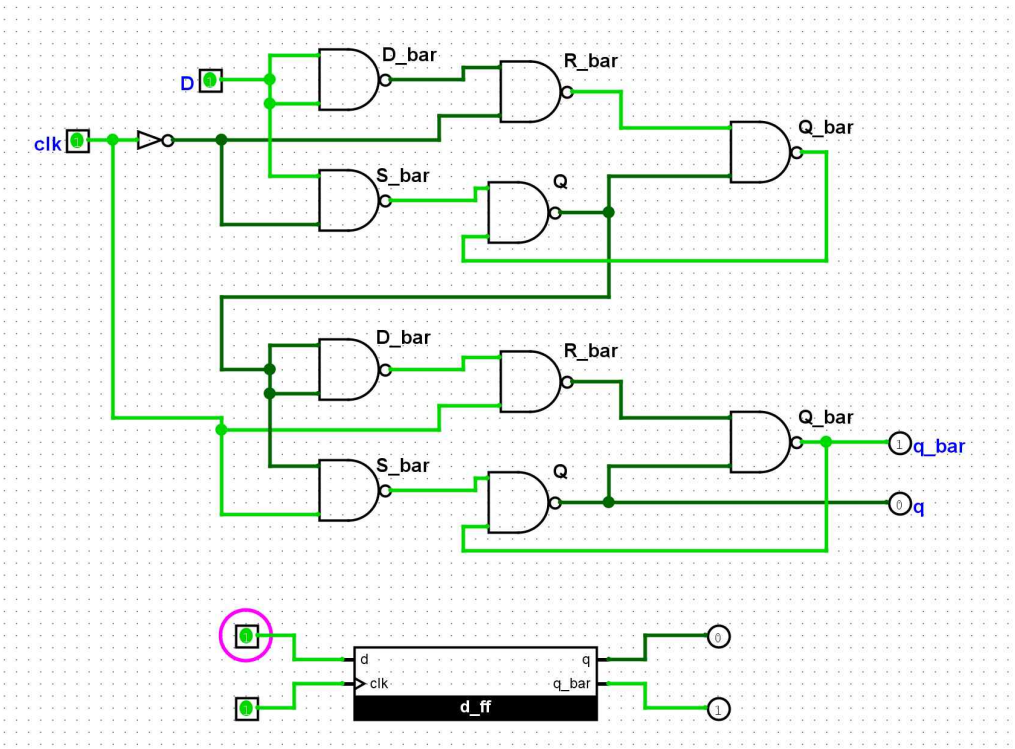


그림 3-2. clk=1 유지, D=1로 변경 → q=0 유지. 에지가 없으므로 출력 불변

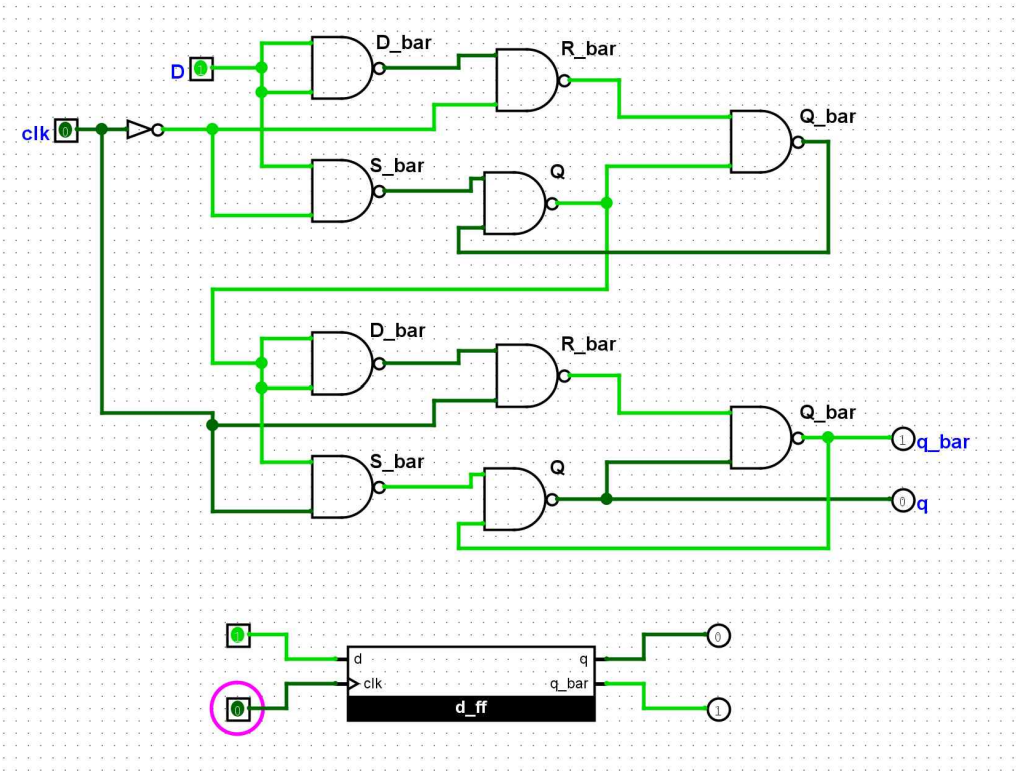


그림 3-3. clk 1→0 → q=0 유지. master만 D=1을 받아들이고 출력은 그대로

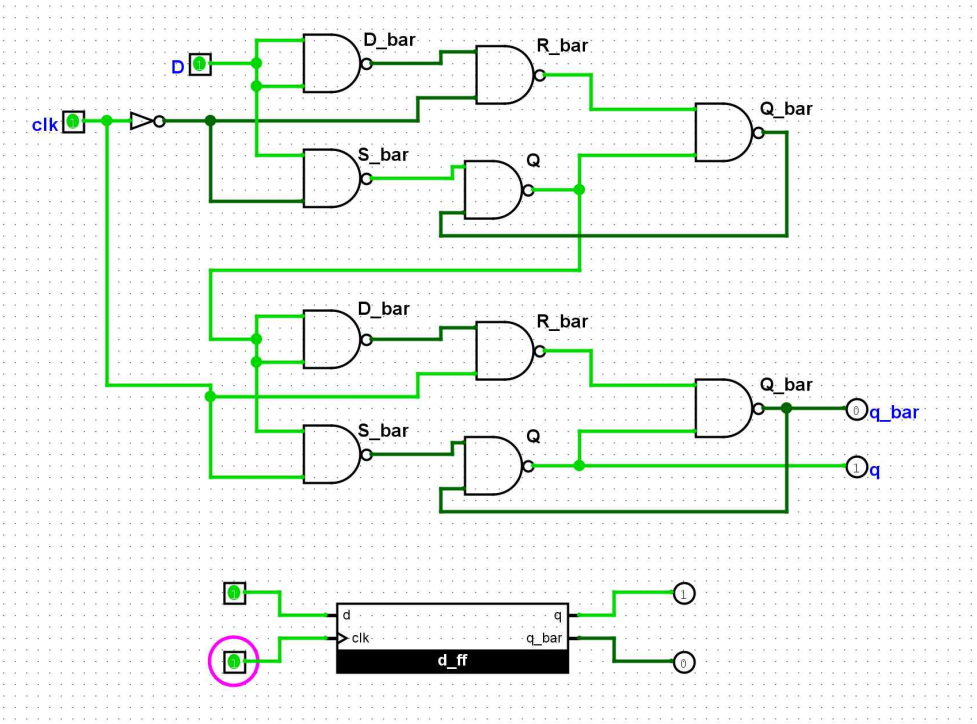


그림 3-4. D=1 상태에서 clk 0→1 → q=1. 상승 에지에서 D=1이 출력으로 전달

D 래치와 달리 clk=1인 동안 D를 바꾸어도 출력이 변하지 않고, clk가 0→1로 바뀌는 순간의 D만 출력으로 옮겨졌다. 두 구현의 출력이 네 단계 모두 일치하였다.

■ Questa 컴파일과 시뮬레이션

같은 d_ff.vhd 를 Questa Altera Starter FPGA Edition 2025.2 에서 vcom 으로 컴파일한 뒤, vsim 에서 force 명령으로 Logisim 실험과 같은 순서의 입력을 주고 파형을 확인하였다. 아래는 Transcript 에 입력한 시뮬레이션 스크립트이다.

[d_ff_sim.do]

```
add wave sim:/d_ff/d sim:/d_ff/clk sim:/d_ff/q sim:/d_ff/q_bar
force clk 0
force d 0
run 10 ns
force clk 1
run 10 ns
force d 1
run 10 ns
force clk 0
run 10 ns
force clk 1
run 10 ns
wave zoom full
```

```
VSIM 81> vcom {C:\VHDL_Assignment\test2\d_ff.vhd}
# Questa Altera Starter FPGA Edition-64 vcom 2025.2 Compiler 2025.05 May 31 2025
# Start time: 09:46:18 on Nov 11,2026
# vcom -reportprogress 300 C:\VHDL_Assignment\test2\d_ff.vhd
# ** Note: (vcom-220) 'C:/Users/gahyu/AppData/Local/quartus/Q1_rs_latch.mpf' is used as the ini file.
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity d_ff
# -- Compiling architecture rtl of d_ff
# End time: 09:46:18 on Nov 11,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

그림 3-5a. d_ff.vhd 컴파일 결과. Errors: 0, Warnings: 0 으로 오류 없이 컴파일됨

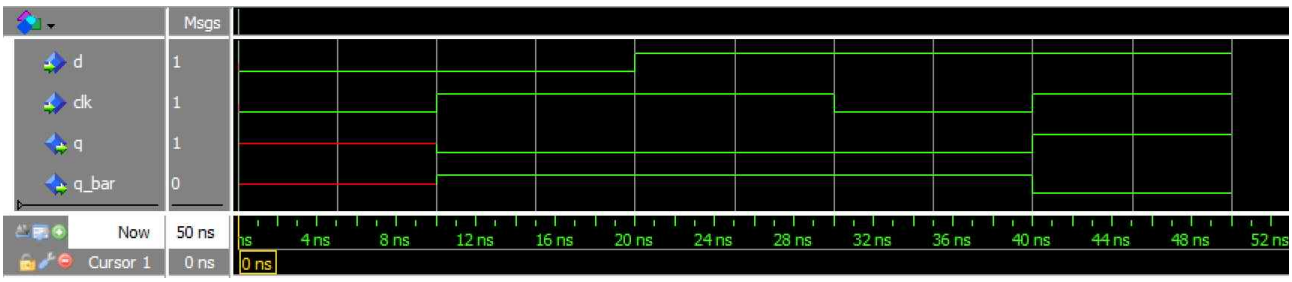


그림 3-5b. Questa 파형. 위에서부터 d, clk, q, q_bar. 각 구간은 10 ns

구간	입력	q	확인한 동작
0~10 ns	clk=0, d=0	U	아직 에지가 없어 초기값(미정, 빨간 선) 유지
10 ns	clk 0→1, d=0	0	상승 에지에서 d=0 저장 (그림 3-1)
20~30 ns	clk=1, d=1	0	에지가 없으므로 d를 바꿔도 불변 (그림 3-2)
30~40 ns	clk 1→0	0	하강 에지에서는 변화 없음 (그림 3-3)
40 ns	clk 0→1, d=1	1	상승 에지에서 d=1 저장 (그림 3-4)

q는 clk가 0→1로 바뀌는 10 ns, 40 ns 에서만 바뀌었고 그 사이 d 변화는 무시되었다. 그림 3-1 ~ 3-4의 Logisim 결과와 같은 에지 트리거 동작이다. 0~10 ns 구간의 빨간 선은 아직 한 번도 저장하지 않은 초기 상태(U)로, Logisim의 초기값 0과 표현만 다르다.

5. JK 플립플롭 (jk_ff)

D 플립플롭 앞에 NAND 게이트로 d_next = J·Q' + K'·Q 논리를 붙이고 출력을 입력으로 피드백하여, 상승 에지마다 유지/Reset/Set/반전을 수행하는 플립플롭이다.

■ VHDL 코드

[jk_ff.vhd]

```
library ieee;
use ieee.std_logic_1164.all;

entity jk_ff is
    port (
        j, k, clk : in std_logic;
        q, q_bar : out std_logic
    );
end jk_ff;

architecture rtl of jk_ff is
    signal qi : std_logic;
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if j = '0' and k = '0' then
                null;
            elsif j = '0' and k = '1' then
                qi <= '0';
            elsif j = '1' and k = '0' then
                qi <= '1';
            elsif j = '1' and k = '1' then
                qi <= not qi;
            end if;
        end if;
    end process;

    q <= qi;
    q_bar <= not qi;
end rtl;
```

■ 코드 설명

- rising_edge(clk) 안에서 J, K 조합 네 가지를 if-elsif로 나누어 기술하였다.
- 00은 null로 이전 값 유지, 01은 Reset, 10은 Set, 11은 qi <= not qi 로 반전(Toggle)한다.
- 게이트 회로의 d_next 논리식을 조건문으로 풀어 쓴 것이며 동작은 같다.

■ 회로도와 실행 결과

회로도 위쪽은 k_bar, A = NAND(J, q_bar), B = NAND(k_bar, q), d_next = NAND(A, B)를 만드는 입력 논리와 그 뒤의 Master-Slave D 플립플롭, 그리고 q, q_bar가 다시 입력 논리로 되돌아가는 피드백 선으로 이루어져 있다. 아래쪽은 jk_ff VHDL 블록이다. J, K를 바꾼 뒤 clk를 0→1로 올리며 출력을 확인하였다.

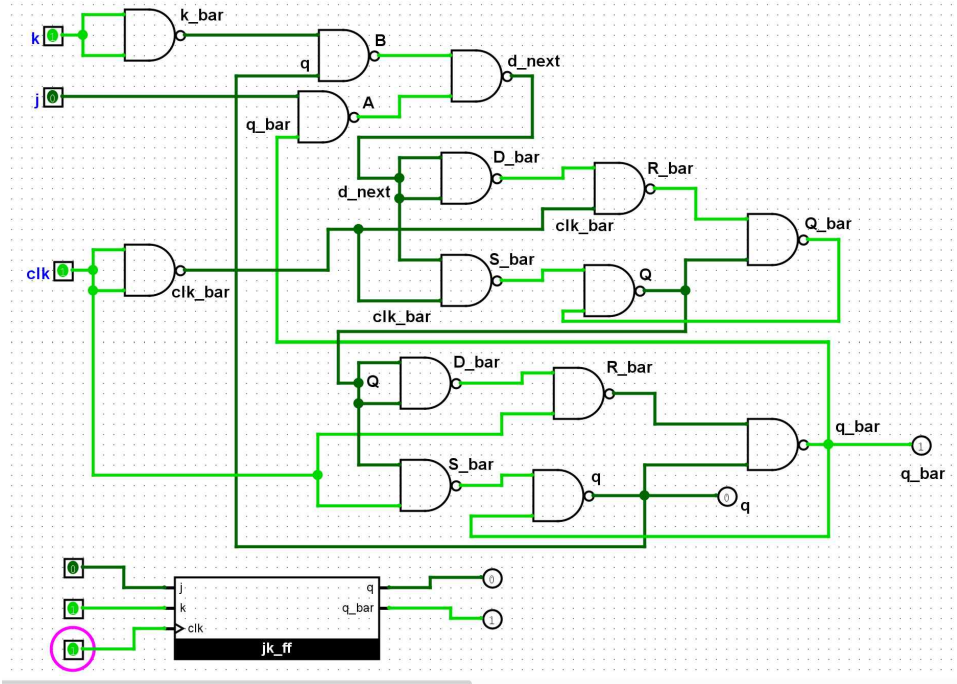


그림 4-1. J=0, K=1, clk 상승 에지 → q=0 (Reset)

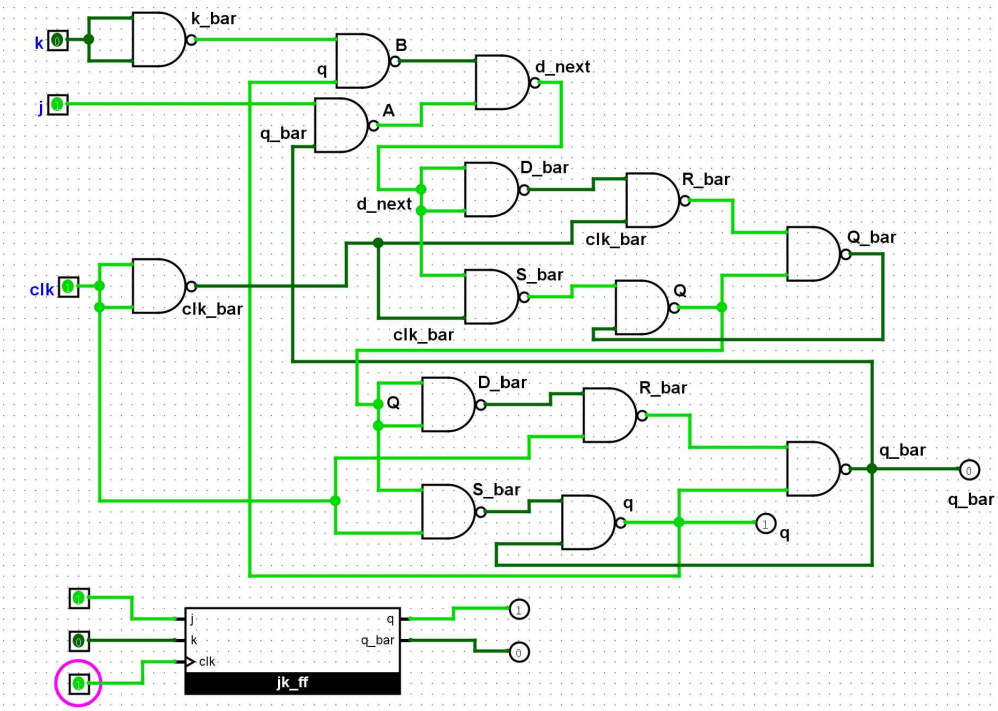


그림 4-2. J=1, K=0, clk 상승 에지 → q=1 (Set)

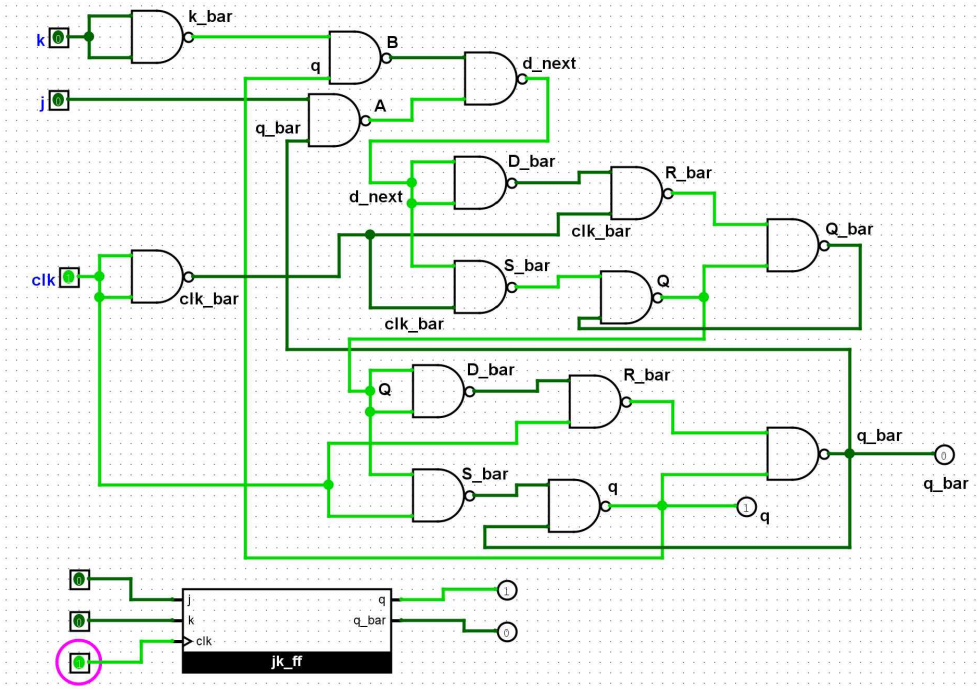


그림 4-3. J=0, K=0, clk 상승 에지 → q=1 유지 (Hold)

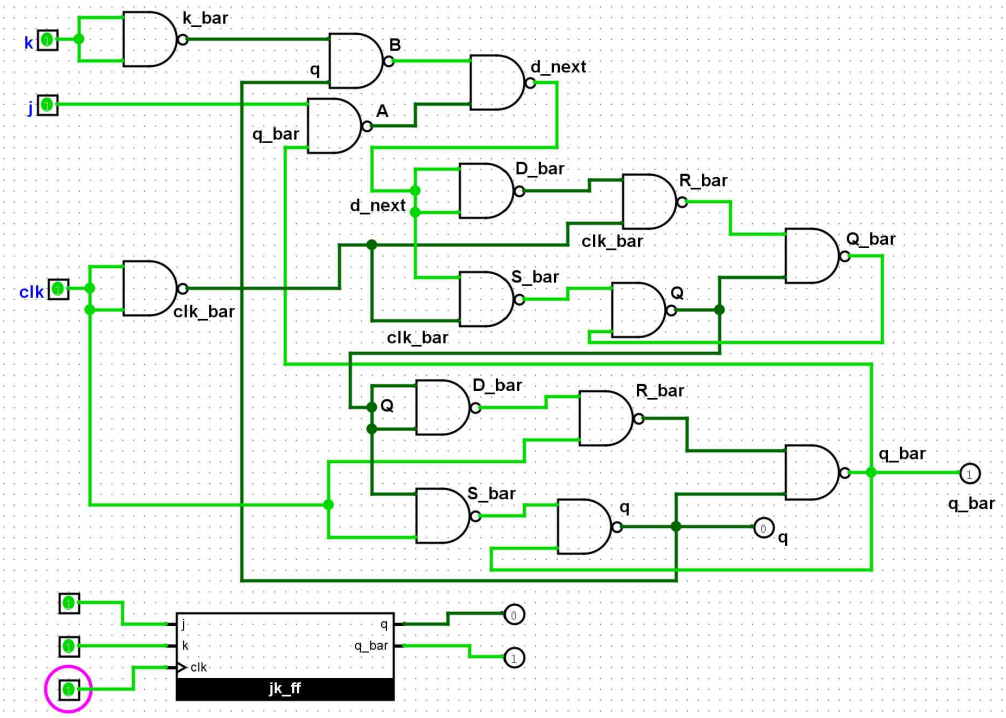


그림 4-4. J=1, K=1, clk 상승 에지 → q=0 (Toggle, 1→0)

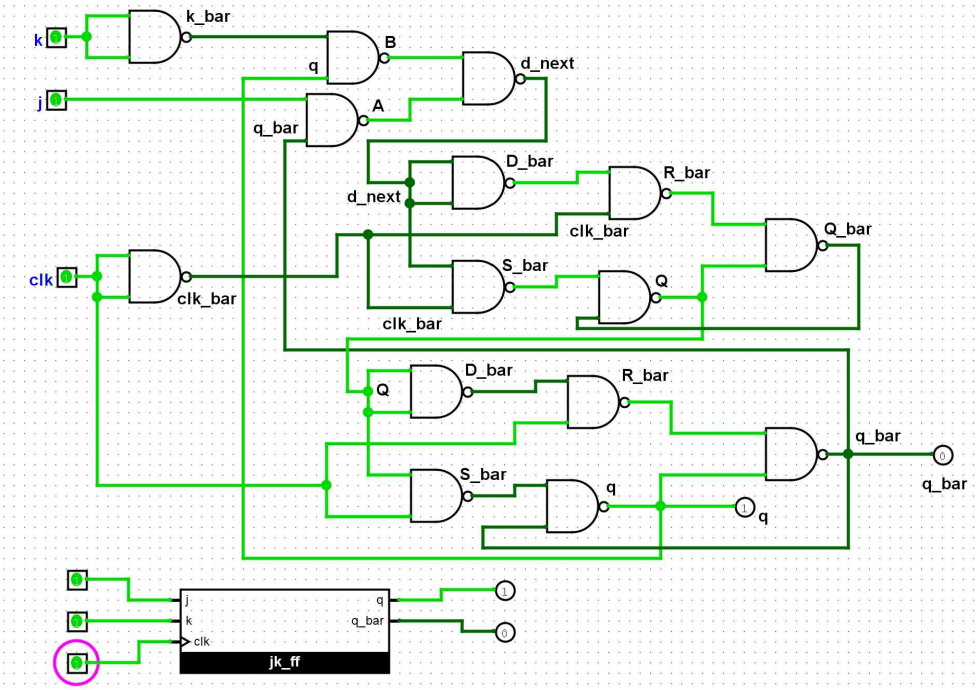


그림 4-5. J=1, K=1, 다음 clk 상승 에지 → q=1 (Toggle, 0→1)

Reset, Set, Hold, Toggle이 모두 클록 상승 에지에서만 일어났고, J=K=1에서 에지마다 출력이 반전되었다. 게이트 회로와 VHDL 블록의 출력이 다섯 단계 모두 일치하였다.

■ Questa 컴파일과 시뮬레이션

같은 jk_ff.vhd 를 Questa Altera Starter FPGA Edition 2025.2 에서 vcom 으로 컴파일한 뒤, vsim 에서 force 명령으로 Logisim 실험과 같은 순서의 입력을 주고 파형을 확인하였다. 아래는 Transcript 에 입력한 시뮬레이션 스크립트이다.

[jk_ff_sim.do]

```
add wave sim:/jk_ff/j sim:/jk_ff/k sim:/jk_ff/clk sim:/jk_ff/q sim:/jk_ff/q_bar
force clk 0
force j 0
force k 1
run 10 ns
force clk 1
run 10 ns

force clk 0
force j 1
force k 0
run 10 ns
force clk 1
run 10 ns
```

```
force clk 0
force j 0
force k 0
run 10 ns
force clk 1
run 10 ns

force clk 0
force j 1
force k 1
run 10 ns
force clk 1
run 10 ns

force clk 0
run 10 ns
force clk 1
run 10 ns
wave zoom full
```

```
VSIM 114> vcom {C:\VHDL_Assignment\test2\jk_ff.vhd}
# Questa Altera Starter FPGA Edition-64 vcom 2025.2 Compiler 2025.05 May 31 2025
# Start time: 09:49:17 on Nov 11,2026
# vcom -reportprogress 300 C:\VHDL_Assignment\test2\jk_ff.vhd
# ** Note: (vcom-220) 'C:/Users/gahyu/AppData/Local/quartus/Q1_rs_latch.mpf' is used as the ini file.
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity jk_ff
# -- Compiling architecture rtl of jk_ff
# End time: 09:49:17 on Nov 11,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

그림 4-6a. jk_ff.vhd 컴파일 결과. Errors: 0, Warnings: 0 으로 오류 없이 컴파일됨

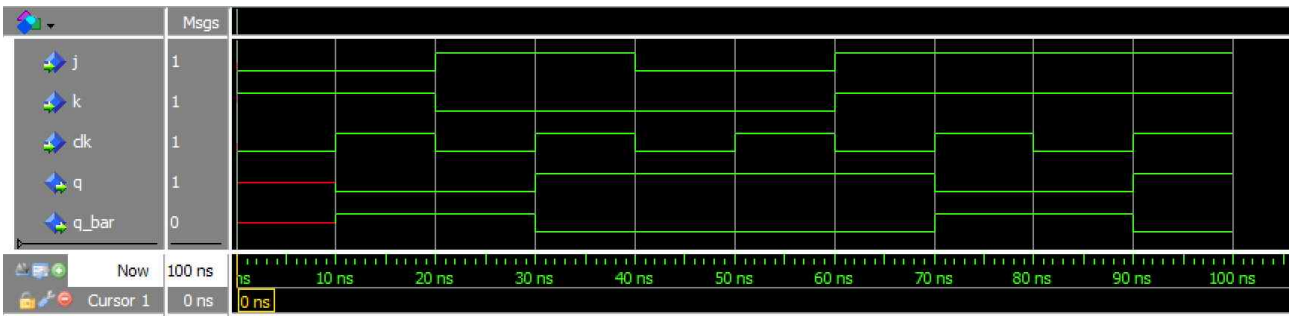


그림 4-6b. Questa 파형. 위에서부터 j, k, clk, q, q_bar. 각 구간은 10 ns

구간	입력	q	확인한 동작
10 ns	J=0, K=1, clk 0→1	0	Reset (그림 4-1)
30 ns	J=1, K=0, clk 0→1	1	Set (그림 4-2)
50 ns	J=0, K=0, clk 0→1	1	Hold. 이전 값 유지 (그림 4-3)
70 ns	J=1, K=1, clk 0→1	0	Toggle. 1→0 반전 (그림 4-4)
90 ns	J=1, K=1, clk 0→1	1	Toggle. 0→1 반전 (그림 4-5)

q는 상승 에지 시점(10, 30, 50, 70, 90 ns)에서만 바뀌었고 Reset, Set, Hold, Toggle, Toggle 순서로 동작하였다. 그림 4-1 ~ 4-5의 Logisim 결과와 모두 일치한다.

6. 비동기 Set/Clear D 플립플롭 (d_ff_async)

3번 D 플립플롭에 클록과 관계없이 출력을 즉시 바꾸는 set, clr 입력을 추가한 플립플롭으로, 둘 다 1이면 Clear가 우선한다.

■ VHDL 코드

[d_ff_async.vhd]

```
library ieee;
use ieee.std_logic_1164.all;

entity d_ff_async is
    port (
        d, clk, set, clr : in std_logic;
        q, q_bar : out std_logic
    );
end d_ff_async;

architecture rtl of d_ff_async is
    signal qi : std_logic;
begin
    process(clk, set, clr)
    begin
        if clr = '1' then
            qi <= '0';
        elsif set = '1' then
            qi <= '1';
        elsif rising_edge(clk) then
            qi <= d;
        end if;
    end process;

    q <= qi;
    q_bar <= not qi;
end rtl;
```

■ 코드 설명

- process(clk, set, clr)로 set과 clr를 감지 목록에 넣어 클록 없이도 즉시 반응한다.
- if clr = '1' 을 가장 먼저 검사하므로 set과 clr가 동시에 1이면 Clear가 우선된다.
- 둘 다 0일 때만 elsif rising_edge(clk) 로 내려가 일반 D 플립플롭처럼 동작한다.

■ 회로도와 실행 결과

회로도 위쪽은 master, slave 두 단의 D 플립플롭에 ASYNC = set OR clr, SET_eff = set AND clr_bar 경로를 추가한 구조이다. ASYNC가 1이면 클록 동기 경로(SD, RD)가 차단되고 S_bar,

R_bar가 set/clr에 의해 직접 결정된다. 아래쪽은 d_ff_async VHDL 블록이다. 먼저 클록을 0으로 둔 채 clr, set만 바꾸어 비동기 동작을 보고, 이어서 둘 다 0으로 내린 뒤 클록 동작을 확인하였다.

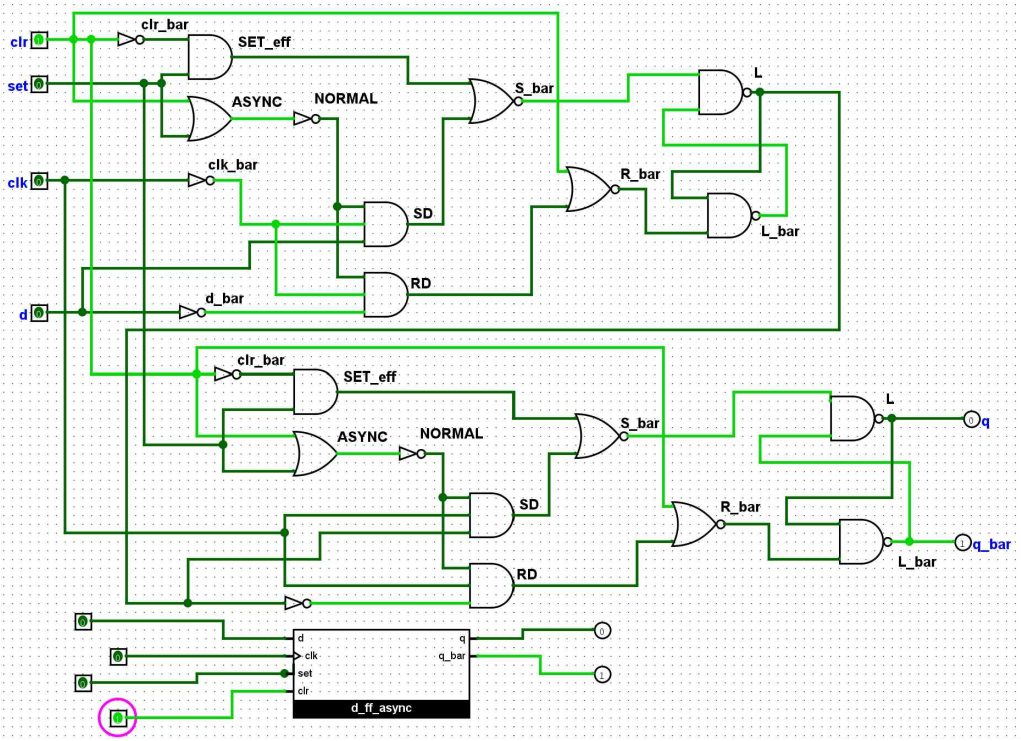


그림 5-1. clr=1, set=0, clk=0 → q=0 즉시 (비동기 Clear)

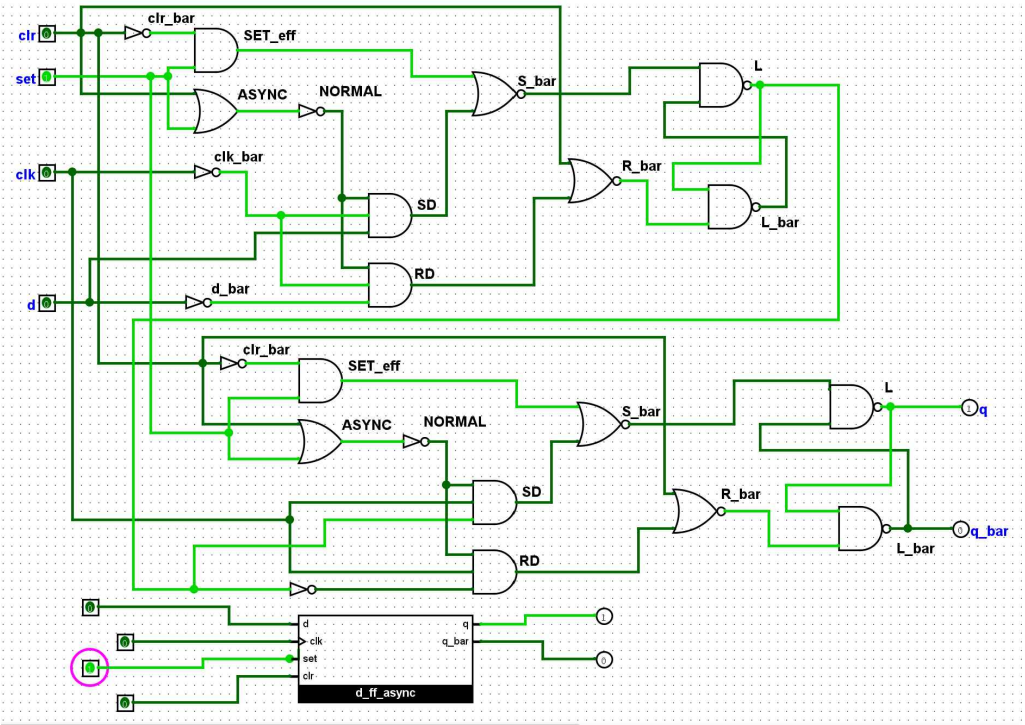


그림 5-2. clr=0, set=1, clk=0 → q=1 즉시 (비동기 Set)

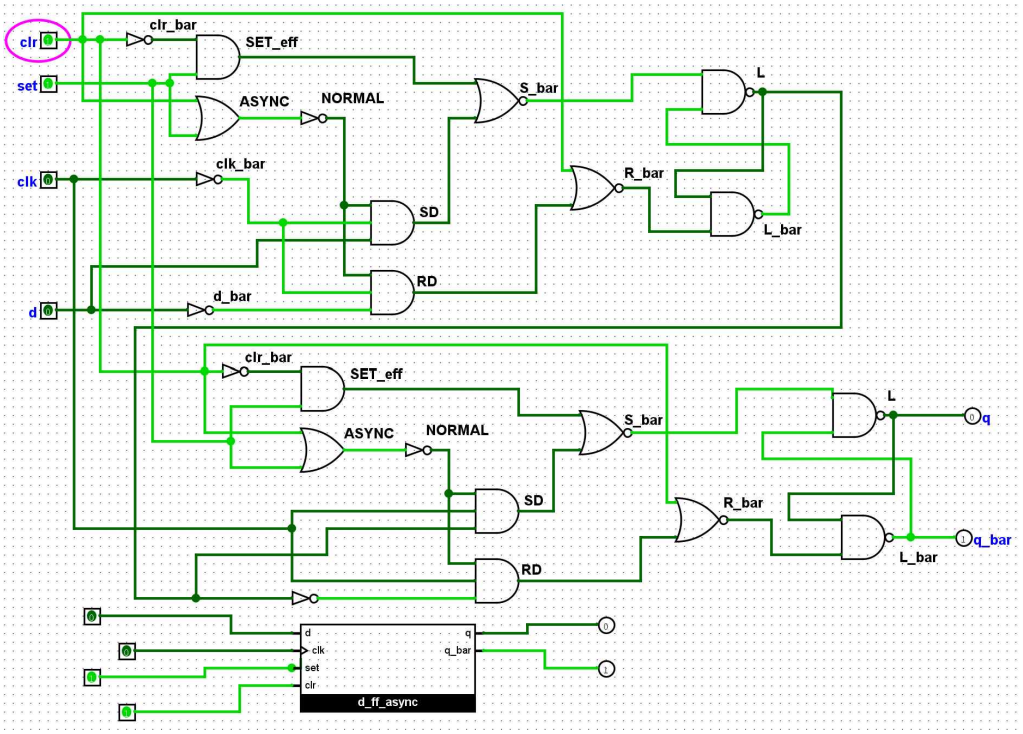


그림 5-3. clr=1, set=1 → q=0 (Clear 우선)

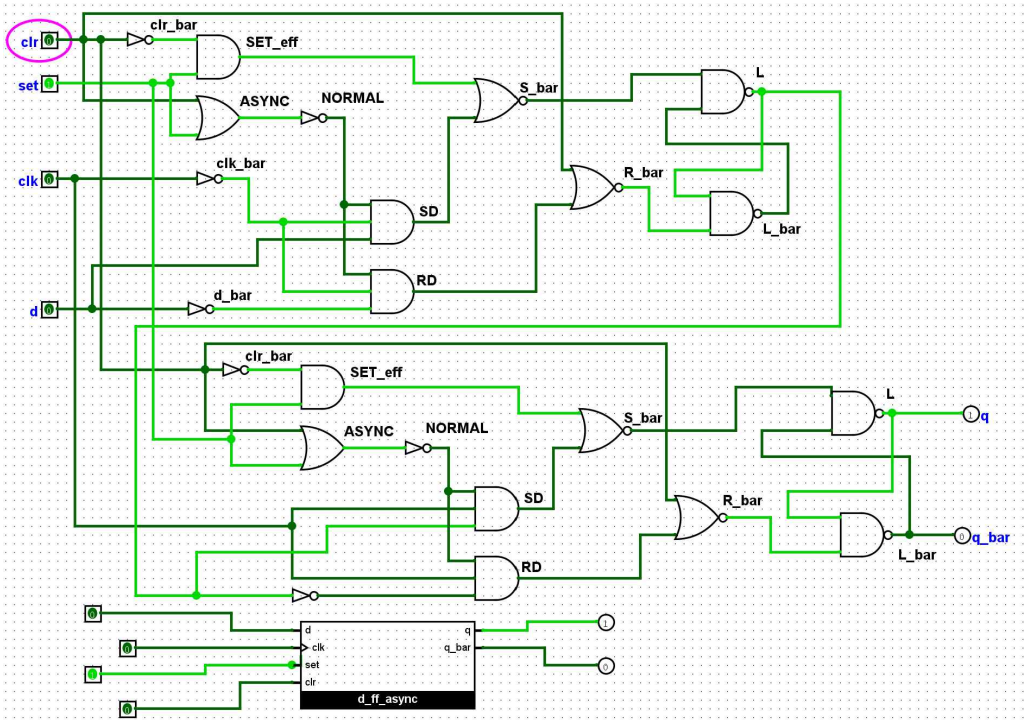


그림 5-4. clr=0, set=1 → q=1 (clr를 내리자 Set이 다시 적용)

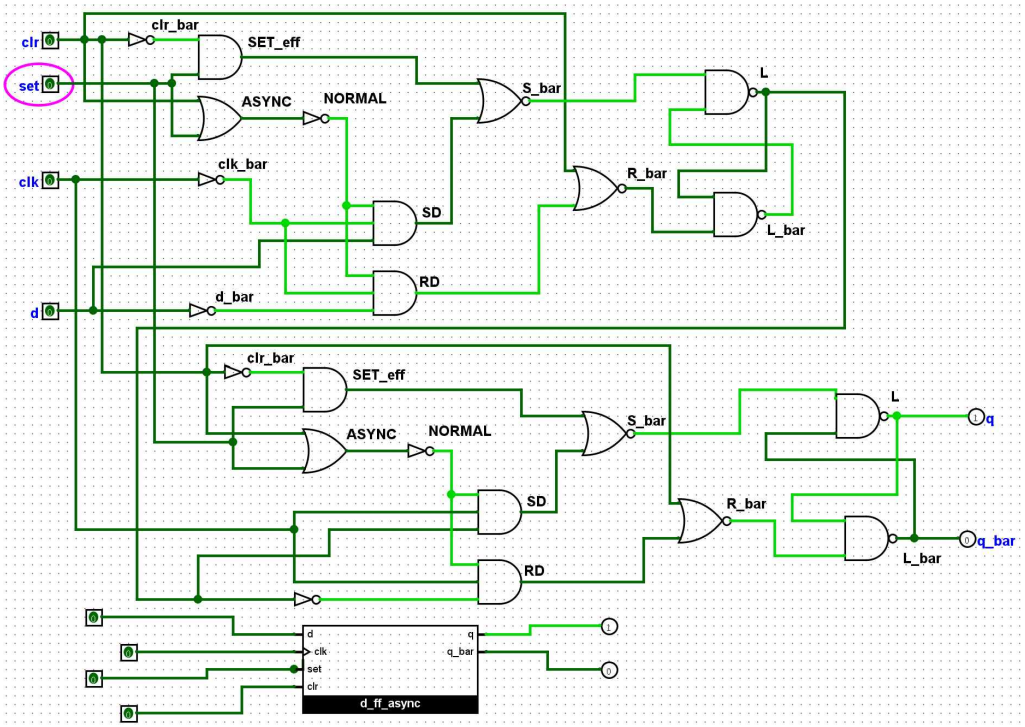


그림 5-5. set=0, clr=0 → q=1 유지 (비동기 입력 해제, 값 보존)

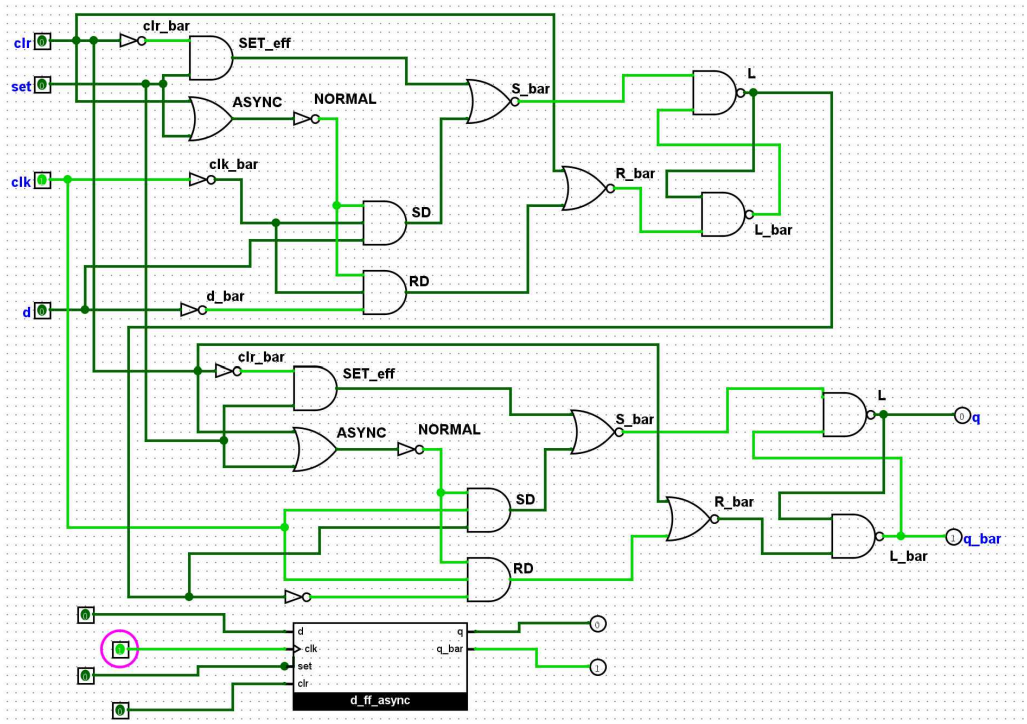


그림 5-6. d=0, clk 0→1 → q=0 (일반 D 플립플롭처럼 상승 에지에서 d 저장)

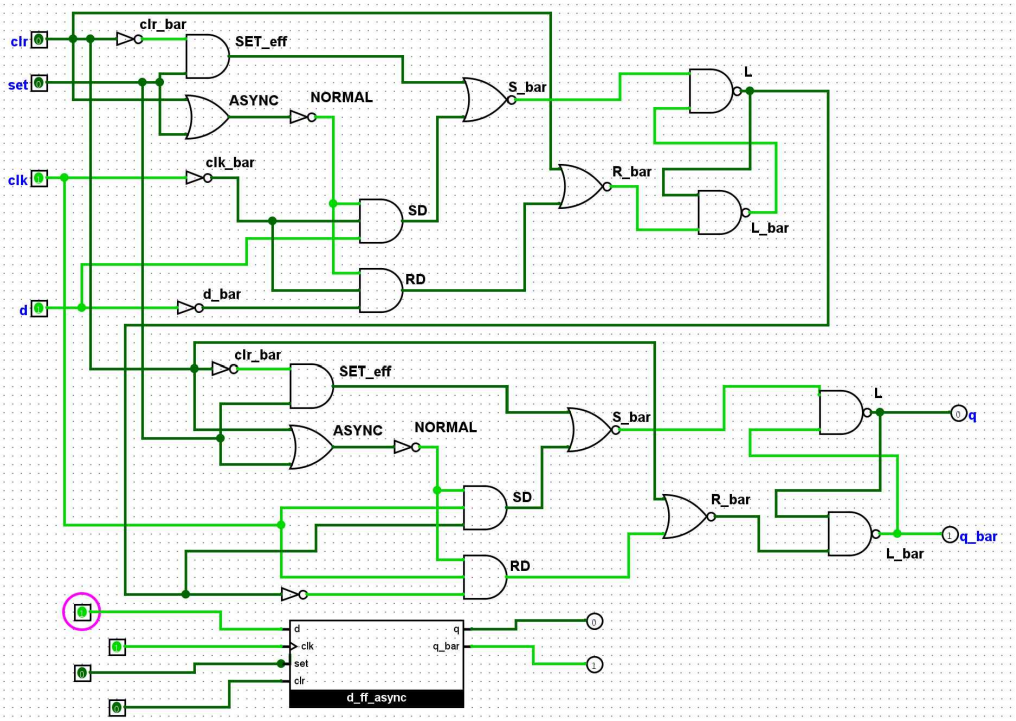


그림 5-7. clk=1 유지, d=1로 변경 → q=0 유지 (에지가 없으므로 불변)

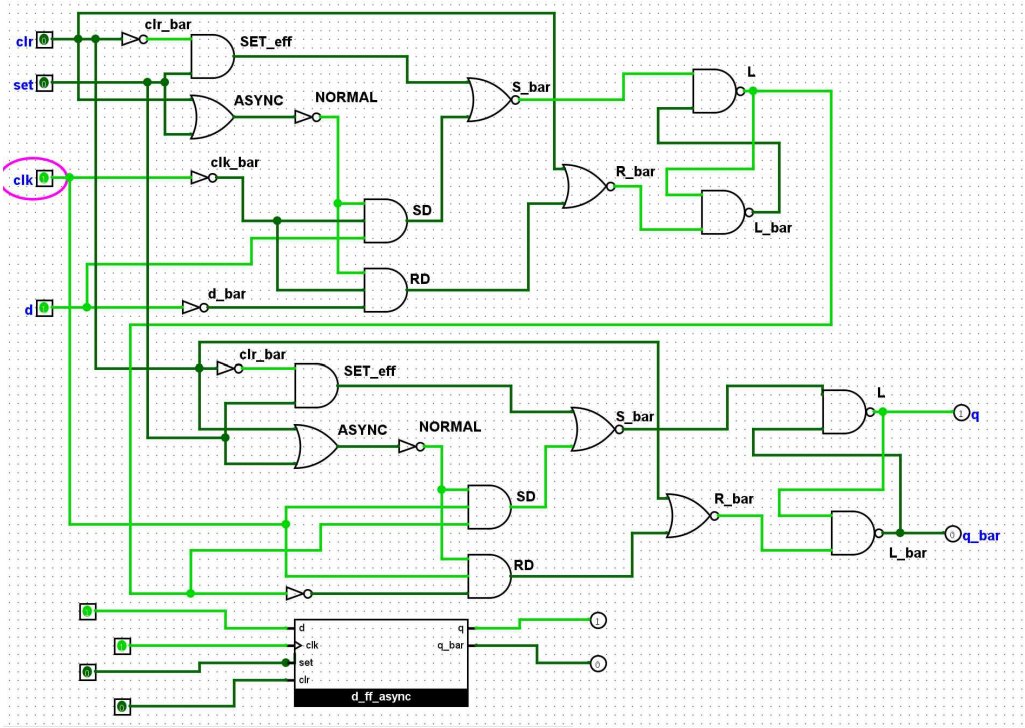


그림 5-8. d=1, clk 0→1 → q=1 (상승 에지에서 d=1 저장)

clr와 set은 클록이 0인 상태에서도 출력을 즉시 바꾸었고 동시에 1이면 Clear가 우선되었다. 비동기 입력을 모두 내린 뒤에는 3번과 같이 상승 에지에서만 d가 저장되었으며, 여덟 단계 모두 두 구현의 출력이 일치하였다.

■ Questa 컴파일과 시뮬레이션

같은 d_ff_async.vhd 를 Questa Altera Starter FPGA Edition 2025.2 에서 vcom 으로 컴파일한 뒤, vsim 에서 force 명령으로 Logisim 실험과 같은 순서의 입력을 주고 파형을 확인하였다. 아래는 Transcript 에 입력한 시뮬레이션 스크립트이다.

[d_ff_async_sim.do]

```
add wave sim:/d_ff_async/d sim:/d_ff_async/clk sim:/d_ff_async/set sim:/d_ff_async/clr
sim:/d_ff_async/q sim:/d_ff_async/q_bar
force clk 0
force d 0
force set 0
force clr 1
run 10 ns
force clr 0
force set 1
run 10 ns
force clr 1
run 10 ns
force clr 0
run 10 ns
force set 0
run 10 ns
force clk 1
run 10 ns
force d 1
run 10 ns
force clk 0
run 10 ns
force clk 1
run 10 ns
wave zoom full
```

```
VSIM 141> vcom {C:\VHDL_Assignment\test2\d_ff_async.vhd}
# Questa Altera Starter FPGA Edition-64 vcom 2025.2 Compiler 2025.05 May 31 2025
# Start time: 09:51:20 on Nov 11,2026
# vcom -reportprogress 300 C:\VHDL_Assignment\test2\d_ff_async.vhd
# ** Note: (vcom-220) 'C:/Users/gahyu/AppData/Local/quartus/Ql_rs_latch.mpf' is used as the ini file.
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity d_ff_async
# -- Compiling architecture rtl of d_ff_async
# End time: 09:51:20 on Nov 11,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

그림 5-9a. d_ff_async.vhd 컴파일 결과. Errors: 0, Warnings: 0 으로 오류 없이 컴파일됨

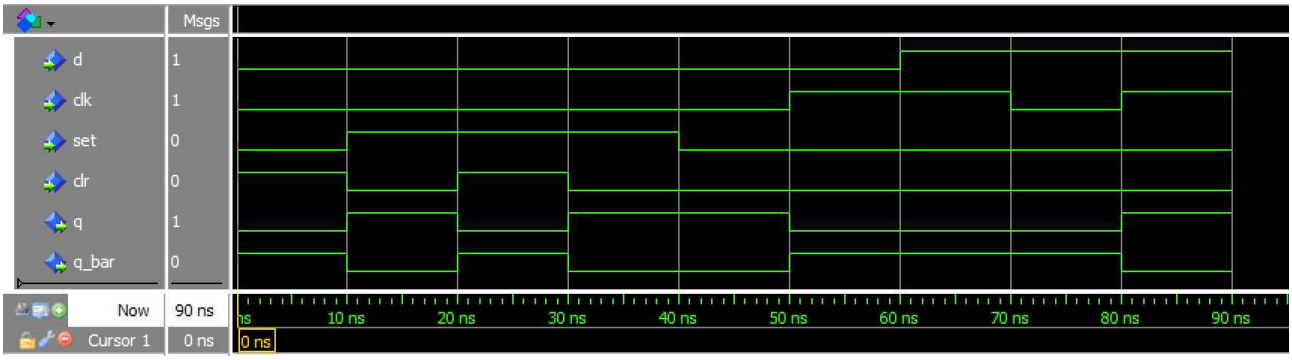


그림 5-9b. Questa 파형. 위에서부터 d, clk, set, clr, q, q_bar. 각 구간은 10 ns

구간	입력	q	확인한 동작
0~10 ns	clr=1, set=0, clk=0	0	비동기 Clear. 클록 없이 즉시 0 (그림 5-1)
10~20 ns	clr=0, set=1	1	비동기 Set. 즉시 1 (그림 5-2)
20~30 ns	clr=1, set=1	0	둘 다 1이면 Clear 우선 (그림 5-3)
30~40 ns	clr=0, set=1	1	clr를 내리자 Set이 다시 적용 (그림 5-4)
40~50 ns	set=0, clr=0	1	비동기 입력 해제, 값 보존 (그림 5-5)
50 ns	clk 0→1, d=0	0	일반 D 플립플롭처럼 에지에서 d=0 저장 (그림 5-6)
60~70 ns	clk=1, d=1	0	에지가 없으므로 불변 (그림 5-7)
80 ns	clk 0→1, d=1	1	에지에서 d=1 저장 (그림 5-8)

clr, set은 clk=0인 상태에서도 q를 즉시 바꾸었고 동시에 1이면 Clear가 우선되었으며, 둘을 내린 뒤에는 상승 에지에서만 d가 저장되었다. 여덟 구간 모두 그림 5-1 ~ 5-8의 Logisim 결과와 일치한다.

7. 종합 비교

각 회로를 세 가지 방법으로 구현·실험한 결과를 정리하면 아래와 같다. 모든 회로에서 Logisim 게이트 회로, Logisim VHDL 블록, Questa 시뮬레이션의 출력이 같은 입력 순서에 대해 동일하게 나타났다.

회로	Logisim 게이트 회로	Logisim VHDL 블록	Questa (vcom/vsim)	결과
RS 래치	NOR 2개 교차 결합	rs_latch.vhd	Errors 0, 5구간 파형	일치
D 래치	NAND 4개	d_latch.vhd	Errors 0, 5구간 파형	일치
D 플립플롭	D 래치 2개 + NOT (Master-Slave)	d_ff.vhd	Errors 0, 상승 에지 2회	일치
JK 플립플롭	NAND 입력 논리 + Master-Slave + 피드백	jk_ff.vhd	Errors 0, 상승 에지 5회	일치
비동기 Set/Clear D FF	D FF + set/clr 비동기 경로	d_ff_async.vhd	Errors 0, 8구간 파형	일치

- 게이트 회로는 피드백 선으로 기억 기능을 만들고, VHDL은 같은 기능을 동시 할당문(RS 래치) 또는 process 안의 if 조건(나머지)으로 기술한다.
- Questa 파형에서 플립플롭의 초기값이 U(빨간 선)로 표시되는 것은 첫 상승 에지 전까지 저장된 값이 없기 때문이며, 첫 에지 이후의 동작은 Logisim과 같다.
- 제출 저장소에는 .vhd 5개, Logisim .circ 5개, Questa 시뮬레이션 스크립트(.do) 5개와 본 보고서 PDF를 함께 올렸다.